

# Chapter 4

## Arrays and Character Strings

### 1. The Array type

A variable of array type is no longer a simple variable, but a complex data structure, grouping information of the same type, accessible by indexes indicating their location. An array can be seen as a group of variables of the same type with the same name.

We distinguish two types of arrays: one-dimensional arrays (Vectors) and multi-dimensional arrays (Matrices).

**Dimension:** The dimension of an array is the number of indices needed to identify a single element.

**Index:** When data is stored in an array, the element is identified by an index which is a non-negative integer ( $\geq 0$ ). The index ranges from 0 to  $N - 1$  (for C language) or from 1 to  $N$  (where  $N$  is the array size).

**Array element:** Elements are items stored in an array and can be accessed by their index.

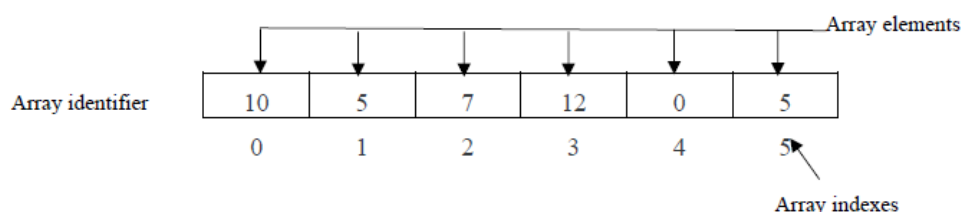
**Array Length:** The length of an array is determined by the number of elements it can contain.

### 2. One-Dimensional Array

It is also referred to as a vector, allowing you to access any of its elements using a single index. Each index value selects a specific element from the array.

#### 2.1. Representation

The array is stored in memory as a sequence of contiguous cells. Once created, new cells cannot be added to or removed from the array (static). The figure below illustrates an array consisting of 8 real numbers.



## 2.2. Declaration

Var array\_name : array [ size] of elementType;

- arrayName: The identifier name given to the array (the variable name).
- Size: The number of elements in the array.
- elementType: The type of elements in the array, which can be of any type like integer (int), float, ...

To declare multiple arrays of the same type, use a comma "," while specifying the size of each array between square brackets [] or ().

### Example

const N=100

Var

marks [N] : **array of** real

tab1[50],tab2[20] : **array of** integer;

## 2.3. Accessing Array Elements

An element from an array is accessed by indexing the array name. This is done by placing the index of the element within square brackets after the name of the array and the expression inside the brackets evaluates to an integer value.

**Syntax to refer to a specific element in the array:**

array\_name[index] ;

### Example

Var Tab : array [8] of integer;

|      | index | 0  | 1 | 2  | 3 | 4 | 5 | 6 | 7  |
|------|-------|----|---|----|---|---|---|---|----|
| Tab= | value | 15 | 7 | 2- | 0 | 9 | 2 | 0 | 3- |

To access the first element of array 'tab', we use tab[0].

To access the fourth element, we use tab[3].

tab[5-3]←tab[tab[3]+1]    ⇔    tab[2]←tab[0+1]    ⇔    tab[2]←7

The array Tab becomes

| Indice | 0  | 1 | 2 | 3 | 4 | 5 | 6 | 7  |
|--------|----|---|---|---|---|---|---|----|
| valeur | 15 | 7 | 7 | 0 | 9 | 2 | 0 | -3 |

**Note:** Accessing an element that doesn't exist (if the index is greater than the array size or negative) will cause the program to terminate.

## 2.4. Assignment

Assigning a value  $x$  to an element  $i$  of a numeric array  $T$  is done by:

$T[i] \leftarrow vx$

For example, the statement:  $T[0] \leftarrow 5$ ; assigns the value 5 to the first element of the array  $T$ .

## 2.5. Reading an Array

To fill an array of  $N$  numbers, we use "read"  $N$  times like:

read (tab[0])

read (tab[1])

...

read (tab[N-1])

However, it's observed that the read instruction is repeated  $N$  times, iterating from 0 to  $N-1$ .

Hence, the "for" loop can be employed with the counter serving as the index.

```
for (I ← 0 to N-1) do
  read(tab[i])
endFor
```

```
or the reverse order
for (i← N-1) to 0
  read(tab[i])
endFor
```

**Example :** Initializing the elements to 0

```
i = 0
```

```
while (i < N-1 Do)
```

```
  T[i] ← 0; i ← i+1 ; endWhile
```

The "read" instruction is used for populating the table, while the "write" instruction is employed to display to the user what is needed.

## 2.6. Displaying an Array

We substitute the read operation with the write operation.

```
write (tab[0])
```

```
write (tab[1])
```

```
...
```

```
write (tab[N-1])
```

```
for (i ← 0 to N-1) do
print(tab[i])
endFor
```

```
or the reverse order
for (i← N-1) to 0
print(T[i])
endFor
```

### Note

- To visit all elements of an array, we generally use the "**for**" loop.
- The array size must be specified during programming (declaration), but we can give the user the impression that the array size can be changed by declaring a large array and using only part of it. We ask the user for the desired size, it must not exceed the actual array size.

## 2.7. Example

Write an algorithm that receives the averages of N students, where N is determined by the user, then calculates the number of students who failed the subject (average less than 10).

```
Algorithm nb_adjourned
var avg :array(100) of real;
i, aj, N : integer
Begin
print ("enter number of students (<", MAX, ")")
read (N)
for (i ← 0 to N-1) do
read (avg[i]);
endfor
aj ← 0
for i ← 0 to N-1 do
  if (avg[i]<10) then
aj ← aj+1;
end if
end for
print("the number of adjourned is ", aj)
End.
```

## 2.8. Array algorithms

### 2.8.1. Searching for elements in an array

Linear search is a straightforward algorithm that traverses the array, comparing each element to the target element. If the target element is found, a Boolean variable is set to true; otherwise, it remains false. Let's consider an array named `tab` with a length of `N`, and an element `x` that we aim to search for:

```
found ← false ; i ← 0 ;
while ((found = false) and (i < N)) do
  if (tab[i] = x) then
    found ← true ;
  endIf ;
  i ← i + 1 ;
endWhile
```

### 2.8.2. Insertion of an element in an array

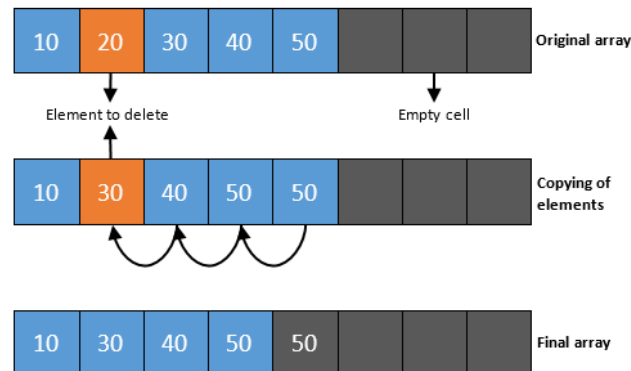
The algorithm which inserts a given value in an unsorted array.

```
// Input the value to be inserted and its position
print("Input the value to be inserted : ");
read(x);
printf("Input the Position, where the value to be inserted :");
read(p);
// Move all data to the right side of the array to make space
for (i ← n to p)
  tab[i] = tab[i - 1];
endFor
// Insert the new value at the given position
tab[p - 1] = x;
// Display the array after insertion
print("After Insert the element the new array is :");
for (i ← 0 to n)
  print(tab[i]);
endFor
```

### 2.8.3. Deleting an element in an array

Logic to remove element from array

1. Move to the specified location which you want to remove in given array.
2. Copy the next element to the current element of array. Which is you need to perform  $\text{array}[i] = \text{array}[i + 1]$ .
3. Repeat above steps till last element of array.
4. Finally decrement the size of array by one.



// Algorithm to delete an element from array at specified position

Algorithm deleting;

Const MAX\_SIZE 100;

var

arr: array [MAX\_SIZE] of integer;

i, size, pos : integer;

Begin

/\* Input size and element in array \*/

print("Enter size of the array : ");

read(size);

print("Enter elements in array : ");

for(i=0 to size) do

read("%d", &arr[i]);

endFor

/\* Input element position to delete \*/

print("Enter the element position to delete : ");

read(pos);

/\* Invalid delete position \*/

if(pos < 0 or pos > size) then

```

    print("Invalid position! Please enter position between 1 to size);
else
/* Copy next element value to current element */
    for(i=pos-1 to i<size-1) do
        arr[i] = arr[i + 1];
    endFor
/* Decrement array size by 1 */
    size= size-1-;
/* Print array after deletion */
    print("Elements of array after delete are : ");
    for(i=0 to size) do
        printf("%d\t", arr[i]);
    endFor
End.

```

#### 2.8.4. Sorting Arrays

Sorting is one of the basic operations on the arrays. Indeed; it's usually helpful when we have an array to be able to put it in some sort of order (be it numerical or alphabetical).

There are many algorithms to sort a list. Some are simple and some are complex. Some are fast while others are slow. We'll look at a few of the most common ones.

##### Selection sort

Selection sort is a simple sorting algorithm that works by repeatedly finding the smallest element in an unsorted array and swapping it with the first element in the array. This process is repeated until the entire array is sorted.

```

for (i ← 0 to (N - 2) do
    smallest = i ;
    for j ← (i + 1) to N-1 do
        if (tab[j] < tab[smallest] )then
            smallest = j;
        endif
    endFor ;
    temp = tab[i]; tab[i] = tab[j]; tab[j] = temp;
endFor

```

## Bubble Sort

Bubble sort is a simple sorting algorithm that works by repeatedly comparing adjacent elements in an array and swapping them if they are in the wrong order. Thus, the smallest element "bubbles" to the top of the array. This process is repeated until the entire array is sorted.

```
for (i ← 0 to n-2 ) do
  for (j ← 0 to n-i-1) do
    if (tab[j]>tab[j+1]) then
      tmp ← tab[j];
      tab[j] ← tab[j+1];
      tab[j+1] ← tmp;
    endif;
  endFor ;
endFor.
```

## 3. Bidimensional array

A two-dimensional array in computer science organizes data into rows and columns, resembling a grid or matrix. It differs from a one-dimensional array by allowing elements to be accessed using two indices: one for the row and one for the column.

### 3.1. Representation

The matrix is represented in memory by a sequence of adjacent cells. A cell cannot be removed or added to the matrix after its creation (static). The following figure represents a matrix with 4 rows and 5 columns of real numbers.

|   | 0  | 1  | 2  | 3  | 4   |
|---|----|----|----|----|-----|
| 0 | 15 | 7  | -3 | 0  | 9   |
| 1 | 6  | 12 | 4  | 33 | 85  |
| 2 | 2  | -8 | 17 | 28 | -52 |
| 3 | 14 | 42 | 36 | 49 | -12 |

### 3.2. Declaration

matrixName: **Array**[Rows][ Columns] **of** elementType;

matrixName`: The identifier given to the matrix (variable name).

Rows`: Number of rows.

Columns`: Number of columns.

elementType`: The type of elements. It can be any type, such as integer, float, ...

The number of elements is the product of the number of rows and the number of columns.

**Example :**

```
const R=100 ;
```

```
const C=100;
```

```
M : Array [R][C] of real ;
```

```
mat1,mat2 : array[30][20] of integer
```

### 3.3. Accessing elements of a matrix

To access a single element of the matrix, we use the matrix name with an index inside two brackets `[` and `]` specifying the row number, and another index inside two brackets `[` and `]` specifying the column number.

To access the element in the first row and first column of matrix `mat`, we use `mat[0][0]`.

.

**Syntax:**

```
array_name [i][j]
```

### 3.4. Reading a matrix

To fill the matrix `M[Rows][Columns]`, we fill in `Rows` rows. Since each row is a one-dimensional array with `Columns` elements.

```
For ( j ← 0 to C-1 ) do
```

```
  read(M[0][j])
```

```
end for
```

```
For ( j ← 0 to C-1 ) do
```

```
  read(M[1][j])
```

```
endfor
```

```
for (j← 0 to C-1 ) do
```

```
  read(M[L-1][j]) ;
```

```
endFor
```

So, we can fill all the matrix using double loop

```
For ( i ← 0 to L -1) do
```

```
  For ( j ← 0 to C-1) do
```

```
    read(M[i][j]);
```

```
  endfor;
```

```
enfor;
```

### 3.5. Displaying a Matrix

Similar to reading, the `write` statement is repeated.

```
for (i← 0 to L-1) do
  for (j← 0 to C-1 ) faire
    print (M[i][j])
  endFor
endFor
```

**Example:** Sum of elements of a matrix

```
algorithm sum_mat ;
const n=4, m=5;
var      mat : array[n-1][m-1] of integer ;
i, j, sum : integer ;
begin
  for (i ← 0 to n -1) do begin
    sum ← 0 ;
    for (j ← 0 to m-1) do
      sum ← sum + mat[i][j] ;
    endFor;
    print(sum);
  endfor;
end
```

## 4. Character String

A character string, or simply a string, is a sequence of characters, like letters, numbers, or symbols that are grouped together. It's commonly used in programming to represent text data. For example, "hello" and "123abc" are both strings.

- A string is a char array containing a null character.
- The null character has 0 for ASCII code and is written '\0'. The significant values of the character string are all those placed before the null character.
- We therefore notice that if the null character is in the first position , we have an empty character string .

## 4.1. Declaration of a string

<NameString> [<sizeMaxString>] : string ;

**In C :**

char < NameString > [<sizeMaxString >] ;

Example

C: array[200] of char;

### Note

We can only place a maximum of 199 letters in 'C', not 200, because we need to include a null character after the last letter of the string.

## 4.2. Initializing a string

The initialization of a string can be achieved either through the conventional array syntax, where the last character indicates the end of the string, or by using quotation marks.

### Example

```
#include <stdio.h>
int main() {
    // Initializing a string using array syntax
    char string1[] = {'H', 'e', 'l', 'l', 'o', '\0'}; // '\0' indicates
the end of the string
    // Initializing a string using quotation marks
    char string2[] = "World";
    printf("String 1: %s\n", string1);
    printf("String 2: %s\n", string2);
    return 0;
}
```

In this example:

- We declare a character array **str** to store the string. It has a size of 100 characters, which means it can store strings up to 99 characters long (plus the null terminator '\0').
- We use **scanf("%s", str)** to read a string from the user and store it in the **str** array. **%s** specifier reads a string until it encounters whitespace (space, tab, newline).

- We use **printf()** to write the string stored in **str** to the console.

### 4.3. Reading and writing a string

During the reading operation, reading of the string stops as soon as it encounters a separator character: newline, space, tabulation, etc. In C, writing a string is done as follows:

```
#include <stdio.h>

int main() {
    char str[100]; // Declare a character array to store the string

    // Reading a string from the user
    printf("Enter a string: ");
    scanf("%s", str); // Read a string from the user and store it in 'str'

    // Writing the string
    printf("You entered: %s\n", str); // Print the string

    return 0;
}
```

### 4.4. Operations on strings in C

Strings can be manipulated like arrays. In this case, we can apply our knowledge of manipulating simple arrays as discussed earlier. Another approach is to use the **string.h** library provided by the compiler, which offers a set of predefined functions to assist in string manipulation.

Operations on strings in C involve various manipulations such as concatenation, comparison, copying, and more. Here's an overview of some common operations:

#### 4.4.1. String Concatenation: Combining two strings into one.

```
#include <stdio.h>
#include <string.h>

int main() {
    char str1[50] = "Hello";
    char str2[50] = " World";
    strcat(str1, str2); // Concatenate str2 to str1
    printf("Concatenated String: %s\n", str1);
    return 0;
}
```

#### 4.4.2. String Comparison: Comparing two strings.

```
#include <stdio.h>
#include <string.h>

int main() {
    char str1[] = "apple";
    char str2[] = "banana";
    int result = strcmp(str1, str2); // Compare str1 and str2
    if (result < 0)
        printf("str1 is less than str2\n");
    else if (result > 0)
        printf("str1 is greater than str2\n");
    else
        printf("str1 is equal to str2\n");
    return 0;
}
```

#### 4.4.3. String Copying: Copying the contents of one string to another.

```
#include <stdio.h>
#include <string.h>

int main() {
    char src[] = "Copy me!";
    char dest[50];
    strcpy(dest, src); // Copy src to dest
    printf("Copied String: %s\n", dest);
    return 0;
}
```

#### 4.4.4. String Length: Finding the length of a string.

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[] = "Hello, World!";
    int length = strlen(str); // Get the length of str
    printf("Length of String: %d\n", length);
    return 0;
}
```

## 5. Practice Exercises

1. Write an algorithm that receives the averages of N students, where N is determined by the user, then calculates the number of students who failed the subject (average less than 10).
2. Write an algorithm that finds the maximum and minimum element in an array of N elements and their indexes.
3. Write an algorithm that finds the sum and the product of all elements in an array
4. Write an algorithm that reads hourly temperatures for 30 days as a matrix (30 by 24), then displays them on the screen. After that, display the highest temperature and when it was recorded.
5. Write an algorithm to add and subtract two matrices. Make sure to handle cases where the matrices have different dimensions.
6. Write an algorithm to find the maximum and minimum values in each row of a given matrix.
7. Write an algorithm to find the string with the maximum and minimum length from a given list of strings. Your program should prompt the user to enter the number of strings in the list and then the strings themselves. After input, the program should determine and display the longest and shortest strings from the list along with their lengths.
8. Write an algorithm to check if a given string is a palindrome. A palindrome is a word, phrase, number, or other sequence of characters that reads the same forward and backward.

radar is palindrome

hello isn't palaindrome